

Is JavaScript fingerprinting still dangerous?

An Empirical Comparison of Browser Fingerprinting Defenses

Ivan Maria Puzelli s348538

Data Protection — Course Project

Abstract—Browser fingerprinting relies on device identifiers being simultaneously *unique* and *stable* over time. Modern defenses break this conjunction via two opposing paradigms: *uniformity*, which standardizes attributes across all users (e.g. Tor Browser), and *randomization*, which injects per session noise (e.g., Brave’s farbling). We present an empirical evaluation across four nested repetition levels, six browser configurations, and two first party domains (with 538 analyzed runs). Proper experimental measurement required patching FingerprintJS 5.2.0, which for instance hardcodes “skipped” for canvas on Gecko, falsely classifying randomizing browsers as perfectly stable. The FingerprintJS library collects data from 42 browser components (43 attributes including the User-Agent header). In our experiment we evaluate the risk of all of them by manually inspecting the possible residual entropy values. For the 9 most used fingerprinting vectors we use the empirical population entropy values taken from different authoritative sources on the matter, since entropy calculations require large scale real world datasets.

Some of these vectors were studied more in detail to understand how different browsers try to protect the fingerprinting surface. For instance we studied in depth Brave’s sparse API boundary mask against canvas fingerprinting: it alters an average of 256.0 pixels, contrasting with Gecko’s total replacement altering 100% of pixels. Weighted by empirical population entropy, Tor removes 90% of tracking surface, Firefox RFP 41%, and Brave 32%. Crucially, entropy reduction measures defensive effort, not tracking outcome: testing surviving bits against user population sizes reveals that only Tor provides an effective anonymity set ($k \approx 1.5 \times 10^5$, enumerated directly), whereas Brave’s residual entropy might still uniquely isolate users ($k \ll 1$ under attribute independence or considering more fingerprinters).

When facing a non trivial adversary that filters out noisy channels and uses more powerful similarity metrics, Farbling browsers might struggle to provide effective protection. Furthermore we examine the “grey sheep” problem, i.e. a fraction of users for which fingerprinting can be even more effective because of an unusual combinations of attributes. Finally we will show that even “naive” and simple JS fingerprinting is still potentially dangerous and only total uniformity can guarantee the users more protection hence more privacy. These naive methods, if well engineered into slightly more complex systems, transform the potential into certainty.

1 INTRODUCTION

Browser fingerprinting identifies a device by querying system configurations through standard JavaScript APIs (such as canvas rendering, installed fonts, and hardware concurrency) without writing a long lived state (such as cookies) to the client. Because it leaves no local artifact, it seamlessly survives cookie deletion, private browsing modes, and the deprecation of third-party cookies [11].

To counter stateless tracking, browser engineers have adopted two fundamentally opposing architectural paradigms even though they are hybrid at some points:

- **Uniformity (Tor Browser):** Standardizes attributes so that all users report identical, shared values, aiming to blend individual devices into a large collective crowd [19] (with Firefox RFP representing a partial desktop implementation).
- **Randomization (Brave):** Injects session or domain specific noise into API responses (farbling), ensuring that a device’s reported values change over time to prevent cross-session linkability [2].

While both approaches claim to defeat tracking, they are built on conflicting assumptions. Uniformity assumes that hiding within a shared crowd eliminates *identifiability*.

Randomization assumes that breaking temporal stability (hence linkability) is sufficient, even if underlying system characteristics remain distinctive. This paper investigates whether these two paradigms are truly equivalent in practice by measuring their behavior under real world tracking scripts (using the modified open source library *FingerprintJS 5.2.0*).

Prior Measurement. Foundational studies about the matter include Panopticlick [7], AmIUnique [14], and large scale web measurements by Gómez-Boix et al. [12], Berke et al. [1] (that uses our same library), and Englehardt and Narayanan [8] which established the widespread deployment of canvas fingerprinting [15]. More recent work has bounded tracker capabilities across different browsers [5], demonstrated practical attacks that recover the original fingerprint despite farbling [17], and how fingerprinting can respawn cleared state [11].

Contributions. We evaluate these defenses on a con-

trolled testbed across multiple browser configurations, repetition levels, and origins:

- **Multi Level Repetition Matrix (§3):** We systematically measure fingerprint changes across four levels of repetition: consecutive reads, page reloads, browser restarts, and visits across different websites to isolate the source of instability and understand how different browsers behave under different conditions (this is to verify Brave browser claims [2]).
- **Measurement Instrument Audit (§3.2):** We show that standard open source fingerprinting tools (FingerprintJS 5.2.0) might require some patching to work properly with modern browsers.
- **Canvas Fingerprinting Deep Dive (§4.2):** We inspect raw canvas pixels in Brave and Gecko based browsers to compare Brave’s sparse API mask with Gecko’s total replacement, evaluating which approach offers better protection against usability costs.
- **Entropy and Anonymity set discussions (§4.4, §4.5):** We price the surviving fingerprinting surface against empirical population benchmarks of previous papers and discuss the implications for whether defenses genuinely hide users in an anonymity set or leave them identifiable.
- **Adaptive Adversaries and Inconsistencies (§4.6, §5):** We analyze how defenses hold up when an adversary ignores noisy channels or exploits similarity metrics, and examine the vulnerability of users with unusual or hybrid attribute combinations. Brave’s claim to “comprehensively defeat FingerprintJS” [2] might hold for the library’s exact hash, not for an adversary who drops the noised channels.

2 BACKGROUND

To evaluate browser fingerprinting defenses rigorously, we must formalize how stateless tracking operates, define the mathematical conditions that enable tracking, and examine how modern browsers implement their respective countermeasures.

2.1 Stateless Tracking

Traditional web tracking relied on cookies, but cookies are fundamentally fragile: they are stored locally on the client machine, easily detected, and routinely cleared by users. Trackers subsequently turned to persistent storage mechanisms such as `localStorage`, `IndexedDB`, and `ETags` (often termed *supercookies*) [11].

Browser fingerprinting represents a categorically different threat because it is entirely *stateless*. Instead of reading stored identifiers from the disk, a script dynamically measures subtle hardware, software, and configuration nuances exposed by standard browser APIs.

Because nothing is stored on the client, fingerprinting completely bypasses cookie clearing, cache flushing, and private browsing modes. Consequently, defensive countermeasures cannot rely on storage isolation; they must actively modify the values the browser *reports* to

Table 1: Applying our definitions to real world browsers (Eq. 1).

	Unique-by-entity	Shared (Non-unique)
Stable	Trackable (Chrome)	Uniformity (Tor Browser)
Unstable	Randomization (Brave)	Ineffective / No Signal

JavaScript.

2.2 Definitions and the Core Asymmetry

We adopt the formal taxonomy of Pugliese et al. [18], which clarifies two distinct notions of uniqueness and one of persistence that are often conflated in the literature. Let $f(e)$ represent the fingerprint of an entity e :

- **Unique-by-appearance:** The fingerprint $f(e)$ appears exactly once within a specific *observed sample* (such as the AmIUnique dataset).
- **Unique-by-entity:** No other entity in the *ground-truth global population* produces the value $f(e)$, even though it may recur across multiple visits by e itself.
- **Stability:** The hardware and software derived core components of $f(e)$ (such as canvas, audio, WebGL, fonts, and screen metrics) remain consistent over time for the same device.

A device is trackable across visits if and only if both conditions hold simultaneously:

$$\text{trackable} \iff \text{unique-by-entity} \wedge \text{stable} \quad (1)$$

The Structural Asymmetry. As summarized in Table 1, defeating a naive tracker requires breaking at least one conjunct of Eq. 1. However, the two defensive paths yield vastly different security guarantees:

1. **Uniformity (Tor Browser)** breaks *uniqueness* while conceding *stability*. Tor’s values are almost always reproducible, with the exception of canvas, but because they are shared identically across (almost) all Tor users worldwide, there will be a set of users emitting an identical fingerprint (the anonymity set). Thus the fingerprint is not unique. Conceding stability over a *shared* value is harmless. An attribute that is shared and stable will be called *pinned* in this paper.
2. **Randomization (Brave)** breaks *stability* while conceding *uniqueness*. Each randomized output is rare or unique, and every attribute that remains unperturbed continues to be both stable and unique (if it’s not *pinned* will be a *leaked* or *randomized* attribute, using our terminology). Conceding uniqueness is dangerous: identifying information survives across all untouched vectors, enabling trackers to recover identity by simply discarding the noisy channels (§4.6). Otherwise they can still apply similarity metrics on the unstable attributes.

Tracking Scopes. Fingerprinting attacks operate at three distinct scopes of increasing adversary capability:

- **First Party Tracking (later L2–L3):** Re identifies a user returning to the same website across reloads and browser sessions.
- **Cross Origin Tracking (later L4):** Correlates visits across distinct domains (`site-a.test` and `site-b.test`) to construct a browsing history profile.
- **Cross Browser Tracking:** Links distinct browsers installed on the same physical host by probing OS and hardware level constants (§4.7).

2.3 Defensive Architectures in Practice

Uniformity: Tor Browser. Tor’s primary design goal is cross origin unlinkability [19]. Rather than injecting noise, Tor enforces strict uniformity by *pinning* browser attributes to universal constants: fonts are restricted to a bundled whitelist, `navigator.languages` is forced to "en-US" (the most common value, which places its users in the largest available crowd), the screen is letterboxed to standardized resolutions, and WebGL reports generic strings. Because Tor users emit identical profiles, an adversary querying standard attributes encounters a large anonymity set.

Randomization: Brave’s Farbling. Brave implements per-session and per-origin randomization known as *farbling* [2] across multiple APIs (canvas, audio, plugins, and hardware concurrency). To perturb API responses, Brave derives a master session domain seed [17]:

$$\begin{aligned} K &= \text{session_key} \oplus \text{domain_key} \\ c &= \text{HMAC-SHA256}(K, \text{api_content}) \end{aligned} \quad (2)$$

where K provides the universal foundation for all farbling. For canvas, the HMAC digest c is computed over the rendered pixels; audio is instead scaled by a per-session, per-site factor in $[0.999, 1]$ [3]. This formulation guarantees three essential properties:

- **(P1) Domain Specific:** Outputs differ across websites, preventing cross origin correlation.
- **(P2) Session Specific:** Outputs change across browser restarts, preventing long term profiling.
- **(P3) In Session Deterministic:** Outputs remain identical within the same page session, preventing visual flickering and application breakage. Crucially, this last property is important because being deterministic within the same session and page reload makes it more difficult for fingerprinters to understand if a browser is using anti-fingerprinting defenses.

Importantly, Brave does not add continuous Gaussian noise(denoising could be done easily). In canvas farbling, the HMAC digest seeds a pseudo random generator that selects a sparse, fixed size set of pixels (§4.2) and flips one color channel of each by strictly ± 1 .

Furthermore, Brave intercepts rendering at the JavaScript API boundary (modifying the pixel buffer

immediately before `toDataURL()` returns), leaving the underlying Chromium rendering engine untouched.

Canvas fingerprinting however does not rely only on the Chromium rendering engine, but is affected also by the underlying hardware and operating system version, hence it is really dangerous for privacy (it can identify the exact version of the OS, the graphics card, the drivers, etc.) [15].

For discrete APIs (such as `hardwareConcurrency`), K directly seeds a pseudo random selector that returns a consistent spoofed value for that domain within a session (any integer between 2 and the true value; we observed 2–8, odd values included).

Canvas Randomization in Gecko. Rather than subtle perturbation, Firefox (under RFP) intercepts canvas readout APIs (`toDataURL`, `getImageData`) [16] and replaces the canvas entirely with pseudorandom bytes drawn afresh at every extraction (§4.2). The rendered canvas is discarded at extraction time, leaving zero host hardware or OS signature in the extracted pixels (at the expense of usability).

Mozilla maintains two distinct fingerprinting protection engines:

1. **privacy.resistFingerprinting (RFP):** Derived from the Tor Uplift project, RFP is an unconditional, high security mode that applies across all websites, combining attribute spoofing with per read canvas randomization at the expense of minor web compatibility. And as we will show, RFP removes a large share of the surface yet yet because of its low usage it might do more damages than the others: the "grey sheep problem".
2. **privacy.fingerprintingProtection (FPP):** A web compatible, contextual defense that remains dormant during standard first party browsing and activates only in private windows or against known tracker lists, less invasive but not totally secure against first party fingerprinting (or supposed first party fingerprinting e.g. CNAME cloaking [6]).

Evaluating Firefox RFP alongside Tor Browser provides an ideal experimental control. Since both browsers share the same Gecko C++ codebase, comparing them proves that defensive code alone is not enough: while Tor achieves safety through complete global uniformity, enabling RFP on an ordinary desktop risks creating a 'grey sheep' profile that mixes spoofed constants with real local system leaks.

2.4 The Fingerprinting Instrument: FingerprintJS

The popular open-source library **FingerprintJS 5.2.0** [10] collects 42 distinct browser components; adding the User-Agent request header gives the 43 attributes we price. To handle anti-fingerprinting defenses, FingerprintJS incorporates a *two draw trap* alongside a hard-coded vendor check:

```
function getCanvasFingerprint():
```

```

if isSafariOrGecko():
    return "skipped" //to avoid canvas defenses
img1 = canvas.toDataURL()
img2 = canvas.toDataURL()
if img1 != img2:
    return "unstable" // Two-draw trap
return hash(img1) // Used in visitorId

```

This logic creates a critical asymmetry between defenses:

- **Tor and RFP are blocked by the vendor check:** Because Tor and RFP run on Gecko, the library immediately hits `isSafariOrGecko()` (in pseudo code) and discards the canvas as "skipped" without even evaluating stability. The check is version gated rather than defense aware: plain Firefox is discarded exactly like RFP and Tor, so the library forfeits canvas for the entire Gecko population. If that check is bypassed, their per read noise ($\text{img1} \neq \text{img2}$) subsequently triggers the two draw trap, returning "unstable".
- **Brave eludes both checks:** Because Brave is Chromium based, `isSafariOrGecko()` evaluates to false. Furthermore, because Brave's farbling is in session deterministic (P3), consecutive reads within the same execution return identical hashes ($\text{img1} == \text{img2}$). FingerprintJS is fooled into treating Brave as a normal, non-defended browser and hashes the farbled canvas into the `visitorId`.

Consequently, unmodified FingerprintJS hashes the remaining 42 static attributes for Tor and RFP, falsely reporting them as 100% stable while hiding their canvas randomization. Accurate measurement therefore required patching both checks (§3.2).

3 METHOD

Brave's three properties are specified over different repetition scopes, so a single stability rate cannot separate them; and the vanilla fingerprinting library, as §3.2 shows, does not report them faithfully.

3.1 Design

Every observation is tagged with one of four nested levels (L1-L4), each isolating one of the properties of the Brave browser (P1-P3) **L1**, within one script execution, reads the canvas twice back to back: raw per-read determinism, probing P3 at its finest grain. **L2**, within one browser process, performs $N = 20$ reloads per domain without quitting the driver: P3. **L3**, across processes, performs $N = 20$ runs per domain, each a fresh process with a fresh profile: P2. **L4**, across domains in one session, visits both in one process: P1. Fresh profiles at L3 are essential, since reusing a profile would carry the session key across runs meant to be independent.

Six configurations were measured: Chrome, Brave (default shields), Brave (aggressive shields), Firefox, Firefox with RFP, and Tor Browser at *Standard*. One Flask process serves two virtual hosts, `site-a.test` and

Table 2: Canvas stability rate at Level 3 for the patched and the unpatched library

Browser	Unpatched	Patched (Raw)
Chrome	1.00	1.00
Brave	0.05	0.05
Brave aggressive	0.05	0.05
Firefox	1.00	1.00
Firefox RFP	1.00	0.05
Tor	1.00	0.05

`site-b.test`, distinguished by the `Host` header, so L4 crosses a genuine origin boundary. Selenium drives all groups, so provenance is uniform. Versions, read from the user-agent of every run: Chrome 151, Brave on Chromium 150, Firefox 147 (both arms), Tor Browser on Firefox 140 ESR. The dataset comprises 540 fingerprints, 538 after the exclusion documented below. Tor *Safer*, *Safest* and Safari are out of scope: Safer withholds the `script` capability from non-HTTPS origins only, so on our plain-HTTP origin it is indistinguishable from Safest and neither is representative of the real HTTPS web.

3.2 Instrument patches

Three modifications to FingerprintJS 5.2.0 were necessary. **M1, raw canvas output:** `getCanvasFingerprint()` returns the literal strings "skipped" (on Gecko 120+) or "unstable" (whenever the two-draw trap fires) in place of pixel data, and a constant is perfectly stable; we export the raw `toDataURL()` of both draws instead. **M2, three-way WebGL outcome:** `shouldAvoidDebugRendererInfo()` returns true for Gecko merely to suppress a console warning, so we always attempt the read and log which of `library-skipped`, `read-succeeded` or `browser-blocked` occurred, where upstream collapses all three into one empty string. **M3, deterministic collection:** we force `load({monitoring: false, delayFallback: 50})`, removing an outbound telemetry beacon and browser-specific idle callback timing as confounds.

Table 2 shows why M1 matters: read through the unpatched library, the two browsers whose entire canvas defense *is* randomization score 1.00, and this finding would have been inverted. The skip is keyed to the engine, so it never fired on Brave; the "two draw" trap never fired on Brave either, because farbling is deterministic within a session and the draws agree (0 of 90 runs differ). The unpatched library therefore measured Brave's canvas correctly and Gecko's not at all.

3.3 Reproducibility

Our automated collection script comprises the server, the probe page, the Selenium driver and one notebook regenerating every table and figure from `results.jsonl`; the three patches are recorded as diffs so the instrument modification can be audited independently of the results. Two decisions affect reported numbers. The canvas used throughout is the raw `toDataURL()` output, never the

Table 3: visitorId stability rate across repetition levels.

Browser	L1 canvas	L2	L3	L4 same-fp
Chrome	1.00	1.00	1.00	1.00
Firefox	1.00	1.00	1.00	1.00
Firefox RFP	0.00	1.00	1.00	1.00
Tor	0.00	1.00	1.00	1.00
Brave	1.00	1.00	0.05	0.00
Brave aggr.	1.00	1.00	0.05	0.00

library’s `canvas` component, since substituting the latter reverses Table 2. And one Level-3 repetition is excluded (two rows): its site-b observation reported a different browser build and locale (Firefox 154, `en-US`) from the other thirty-nine; if we didn’t exclude these rows it would have credited Firefox RFP with randomizing the user agent and languages, which it does not. All measurements come from a single Apple Silicon host in one locale, so the ground truth are specific to it.

4 RESULTS

We evaluate browser defenses across five progressive questions: whether farbling satisfies its theoretical properties (§4.1); how canvas pixel mechanics differ in practice between Brave and Firefox RFP (§4.2); which attributes survive in the empirical entropy budget (§4.4); whether surviving bits fit into realistic browser populations via joint entropy deflation (§4.5); and how defenses withstand an adaptive adversary (§4.6–4.7).

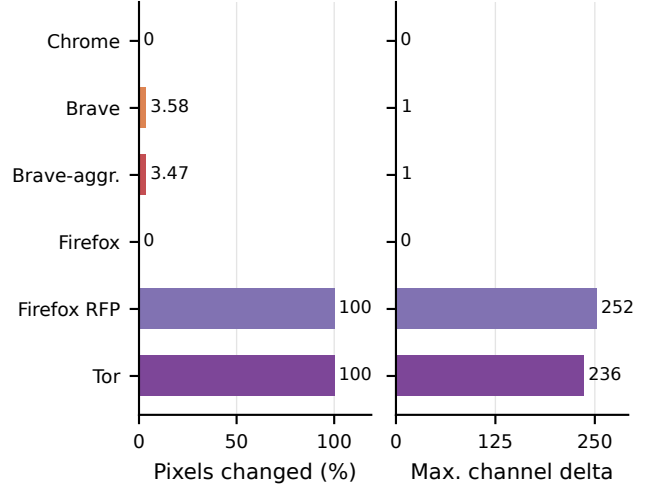
4.1 Per-Read Determinism and the Three Properties

Brave’s farbling model asserts that API responses remain deterministic within a single session (P3). In contrast, fingerprinting libraries like FingerprintJS attempt to detect randomization using a two-draw trap: rendering the canvas twice and flagging disagreement as evidence of spoofing.

While two consecutive draw divergence is a sufficient condition to detect per read randomization, it is not necessary to prove farbling. Because Brave is deterministic within a session, its two draws agree identically (90/90 runs return “measured”), evading the trap completely. Conversely, the trap fires on every single run of Firefox RFP (0/88, the denominator reduced by the exclusion documented in §3) and Tor (0/90), revealing per read randomization. Paradoxically, per read randomization makes RFP and Tor trivially detectable to a tracker, whereas Brave’s per session determinism avoids detection.

As shown in Table 3, Brave successfully satisfies all three properties: L2 = 1.00 confirms in-session determinism (P3); L3 = 0.05 = 1/20 confirms cross-session unlinkability, with twenty fresh browser profiles yielding twenty distinct identifiers (P2); and L4 = 0.00 confirms cross-origin isolation across test sites (P1).

No other browser exhibits this behaviour. Every other configuration, including Tor Browser, returns an *identical* fingerprint to both websites in a single session. This illustrates the core asymmetry from §2.2: uniformity does not

**Figure 1:** Canvas mechanism: Brave changes few pixels by one channel unit; RFP and Tor change every pixel.

produce site-specific values, but emits a globally *shared* signature. Across different domains, Brave shares an average component agreement of 0.905 (Jaccard similarity 0.826), differing only in `canvas`, `audio`, `plugins`, and `hardwareConcurrency` (replaced by `screenResolution` under aggressive shields). Chrome and Firefox maintain 1.000 component agreement; RFP and Tor reach 0.976 when their per-read-changing raw canvas is included.

4.2 Canvas Defense Mechanics

Stability rates only indicate *if* a canvas changed, not *how*. By comparing raw pixel arrays between runs (10 same-domain comparisons against one reference per domain, 20 per arm), we isolate two fundamentally distinct defensive mechanisms (Fig. 1).

Brave alters only 3.58% of pixels across independent sessions, and across all 10,314 altered pixels, every changed color channel changes by strictly ± 1 . Differencing Brave against undefended Chrome isolates a single mask: it alters an average of 256.0 pixels out of 14,400 (1.78%, sd 13.1, range 224–286). All 40 counts are even, as Brave’s code implies: its LFSR visits 512 candidate pixels, each of the 256 HMAC key bits gating two, so the count is $2 \times \text{Binomial}(256, \frac{1}{2})$ (mean 256) [3].

Consequently, *the larger the canvas drawn by a script, the smaller the fraction perturbed by Brave*: a megapixel canvas still receives only ≈ 256 modified pixels (0.02%), leaving the underlying hardware and OS rasterization completely intact everywhere else.

In sharp contrast, Firefox RFP and Tor do not perform subtle perturbation. They do not want to accept the risk of canvas “reverse engineering”. As implemented in Firefox’s canvas code [16], Gecko leaves the internal rendering pipeline untouched but intercepts readout APIs (`toDataURL`, `getImageData`) to perform *total replacement*. The rendered image is discarded at extraction time and replaced with 32 pseudorandom bytes repeated cyclically across the entire image (`pixel[i] = pixel[i (mod 8)]`). This results in 100% altered pixels with maximum color chan-

nel deltas of 252 and 236, respectively, eliminating the host’s hardware signature. While this total replacement eliminates the host signature, it incurs a sharp usability penalty for legitimate applications and makes the defense immediately detectable. Appendix A shows this penalty on a signature pad and a QR ticket: Brave’s exported QR code still scans, whereas Tor’s export is a repeating pattern that does not.

Pixel analysis also confirms Brave’s determinism: 0.00% of pixels change across reloads within a session (P3), while 3.31% change across domains (P1) and 3.58% across sessions (P2), exactly tracking $K = \text{session_key} \oplus \text{domain_key}$. In contrast, Firefox RFP and Tor replace 100% of pixels on every extraction, so pixel differences across reloads, domains, and sessions are identically total.

4.3 What Each Browser Conceals

We evaluate defensive coverage by comparing each protected browser against an undefended baseline of the same engine family, ensuring engine specific defaults are not misattributed as defenses.

Tor enforces *uniform pinning*: timezone is pinned to *Atlantic/Reykjavik* (on an Italian host in *Europe/Rome*), language is forced to *"en-US"*, color depth is clamped to 24, and the window is letterboxed to 600×1200 . Brave enforces *randomization* on canvas, audio, plugins, and CPU cores, but leaves the real system timezone and language exposed.

As shown in the component heatmap (Fig. 2), the two defensive strategies diverge sharply. Brave’s WebGL defense (generic renderer string, randomized extension list) began a phased rollout only in August 2026 [4] and was inactive in our runs: *webGLBasics* and *webGLExtensions* remained completely stable across all 40 Brave sessions, leaking the true hardware GPU renderer (*ANGLE (Apple, ANGLE Metal Renderer: Apple M3)*). On Gecko, Patch M2 reveals that plain Firefox returns *"Apple M1, or similar"*, while RFP and Tor substitute the uniform string *"Mozilla"*.

4.4 The Entropy Budget

To quantify real world tracking risk, we weight attributes using empirical population entropy from AmIUnique [14, 13]. Because evaluating entropy, because of entropy definition is beyond a single host experiment. Hence we borrowed large scale population weights from different experts on the matter, and these weights provided a good approximation.

We evaluate an attribute under a returning visitor model: an attribute contributes its entropy if and only if it is *present*, *stable* across sessions, and *genuine*. If an attribute is randomized across sessions or pinned to a universal constant shared by all users, its tracking payout is zero. From the 43 collected features, we restrict the budget to the 9 high entropy attributes with published empirical population weights. Eight are drawn from the AmIUnique experiment ($N = 118,934$) [14, 13]. Because of the fact that *hardwareConcurrency* post dates

Table 4: Attribute entropy dispositions: rnd and pin attributes are not priced

Attribute	Bits	Chrome	Brave	RFP	Tor
<i>userAgent</i>	9.78	leak	leak	leak	pin.
<i>fonts</i>	8.38	leak	leak	leak	pin.
<i>canvas</i>	8.28	leak	rnd.	rnd.	rnd.
<i>languages</i>	5.92	leak	leak	leak	pin.
<i>screenResolution</i>	4.89	leak	rnd.	pin.	pin.
<i>webGLBasics</i>	3.41	leak	leak	pin.	pin.
<i>timezone</i>	3.34	leak	leak	pin.	pin.
<i>hardwareConcurrency</i> [†]	2.34	leak	rnd.	leak	leak
<i>platform</i>	2.31	leak	leak	leak	leak
Surviving Bits	48.65	48.7	33.1	28.7	4.6
Reduction		0%	32%	41%	90%

that study, its weight (2.34 bits) is sourced from Berke et al. [1] ($N = 8,400$). Since the remaining 34 unpriced attributes are omitted from the sum, our baseline (48.65 bits) 4 and surviving scores represent a principled conservative lower bound on the adversary’s total tracking power across the full fingerprint surface.

Four methodological rules govern these scores:

- 1. Substitutions must be shared (pinned):** Brave reports a Chrome user agent with version 150 rather than the host’s Chrome 151. We treat this minor version lag as release skew rather than a defense, scoring it as leaked. Aligning user agents version however to the dominant Chrome release could maximize crowd size, hence increasing privacy. In contrast, Tor and RFP share identical constants (*Atlantic/Reykjavik*, *"Mozilla"*, color depth 24), which cannot occur by chance.
- 2. Avoiding Redundant Counts:** Tor pins its browser version globally, leaving only the host OS variable in its User Agent. Because the OS is already charged under *platform* (2.31 bits), scoring *userAgent* would count two times the exact same information ($H(\text{UA} \mid \text{platform}) \approx 0$). In contrast, Chrome, Firefox, and RFP also leak patch levels and update skew, so their full 9.78 bits stand as leaked.
- 3. Binary Scoring of Hybrid Defenses:** Our model applies strict binary scoring (0 bits vs. full bits). Brave reports the first of seven common screen sizes that fits the window [3] (1366×768 in 36 of 40 runs, never the host’s true 1280×832). While this channel exhibits low variance, it successfully conceals the host’s physical display. Treating this hybrid between pinning and noise as protected credits Brave zero bits.

Under this model, Tor removes 90% of the tracking surface (leaving 4.65 bits), Firefox RFP removes 41% (28.73 bits), and Brave removes 32% (33.14 bits).

Robustness Across Price Lists. To confirm that these results are not artifacts of a single dataset, Table 5 evaluates our measured attribute dispositions under three

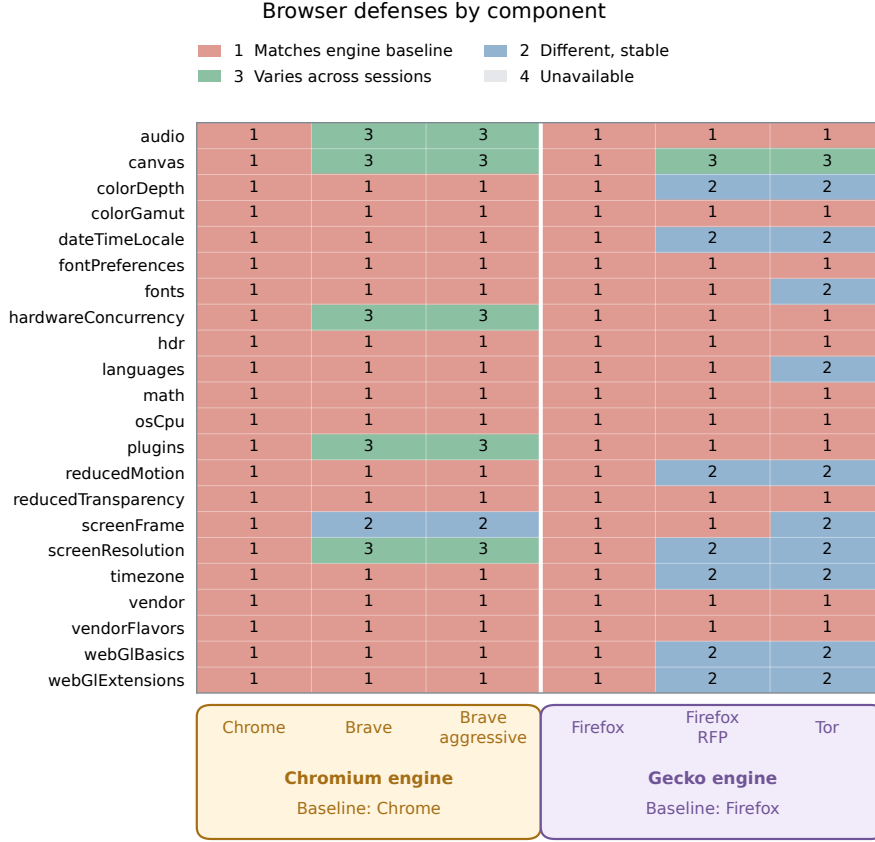


Figure 2: Observed behavior of 22 informative components at Level 3. Blue means different but stable, not proof of population-wide pinning; the 20 omitted components are constant across the tested arms.

Table 5: Robustness across five weighting schemes.

Weighting Scheme	N	Brave	RFP	Tor
Panopticlick (2010)	470k	15%	25%	100%
AmIUnique (2016)	119k	32%	41%	90%
Hiding in the Crowd (2018)	2.07M	36%	52%	97%
Berke et al. (2025)	8.4k	30%	58%	83%
Unweighted (43 attributes)	—	17%	24%	33%

additional literature weightings, as well as an unweighted count of all 43 attributes (where all attributes have the same importance). Across all five schemes, the ranking remains strictly invariant: Tor > Firefox RFP > Brave in terms of privacy and security.

4.5 Does the Residual Fit the Crowd?

Percentage reductions measure *defensive effort*, not *tracking outcome*: removing 41% of entropy is ineffective if the surviving 59% still uniquely identifies the user.

When a browser substitutes attributes, it places the user into a crowd of individuals emitting that same substitution. For surviving entropy not to single the user out within that crowd, the crowd size must exceed $2^{H_{\text{res}}}$. However, summing marginal entropies assumes attribute independence, whereas real world browser attributes correlate heavily (e.g., GPU renderer with OS, screen resolution with device platform). The true information exposed to a

tracker is the *joint entropy* $H(X_1, \dots, X_n) \leq \sum H(X_i)$.

To map marginal residual entropy H_{res} to joint entropy, we apply a correlation deflation factor $\delta \geq 1$, giving effective joint residual entropy H_{res}/δ . We formalize this via the expected *anonymity set size* k :

$$k = \frac{N}{2^{H_{\text{res}}/\delta}} \quad (3)$$

where N is the browser’s active user population. Because browser attributes are physically correlated (e.g., operating system with available fonts), naive marginal addition (48.65 bits) overcounts entropy. In AmIUnique, Laperdrix et al. [14] observed 89.4% uniqueness among $N = 118,934$ users, corresponding to an effective joint entropy of ≈ 17.1 bits (implying a deflation factor $\delta \approx 2.8$). On a larger, mainstream commercial population, Gómez-Boix et al. [12] measured 33.6% uniqueness ($N = 2,067,942$), yielding 20.85 bits of joint entropy ($\delta \approx 2.33$). Cross-study census data [7, 14, 12] thus places the plausible empirical deflation factor δ between 2.3 and 2.8. We therefore evaluate three correlation regimes:

- **Super-pessimistic** ($\delta = 1$): face-value marginal entropy, assuming complete attribute independence (worst-case for privacy).
- **Slightly pessimistic** ($\delta = 2$): conservative defla-

Table 6: Anonymity set capacity (k). N is each browser’s estimated active user base, monthly for every arm except Tor, for which we keep Tor Metrics’ *daily* count so that its k remains a floor rather than a best case.

Browser	N	H_{res}	$k_{\delta=1}$	$k_{\delta=2}$	$k_{\delta=2.5}$
Chrome	3.5×10^9	48.65	8×10^{-6}	167	4,853
Firefox	1.8×10^8	48.65	4×10^{-7}	9	250
Brave	8×10^7	33.14	8×10^{-3}	822	8,178
Firefox RFP	10^6 (assumed)	28.73	2×10^{-3}	47	347
Tor	2×10^6	4.65	1.5×10^5	(enumerated)	

tion accounting for moderate inter-attribute correlation.

- **Slightly optimistic** ($\delta = 2.5$): calibrated to empirical joint entropy measurements in the literature, representing an optimistic case for browser defenses.

Table 6 reveals a fundamental divide between defenses:

- **Tor** dominates and requires no correlation deflation. Unlike browsers exposing dozens of correlated attributes, Tor’s residual surface reduces to only two unpinned variables: `platform` (three desktop OS classes) and `hardwareConcurrency` (four common core counts), which our runs confirm pass through unmodified (`MacIntel`, 8). Under our evaluation ruleset, both are strictly classified as *leaked* because they expose genuine hardware ground truth. However, because the vast majority of users cluster into a few ubiquitous configurations (e.g., Windows on 4 or 8 cores), the adversary’s actual information gain is minimal. We intentionally retain them as leaked to preserve methodological consistency and provide a conservative upper bound on Tor’s residual surface; our manual architectural audit confirms that no other component introduces unaccounted identifying entropy. A direct enumeration partitions Tor’s 2 million active users across merely 12–15 discrete hardware buckets (3.6–3.9 bits of joint entropy). Even in the rarest bucket (Linux on 16 cores, $\approx 0.5\%$ of users), Tor guarantees a minimum anonymity set of $k \approx 10^4$ indistinguishable peers, averaging $k \approx 1.5 \times 10^5$. These values act however as absolute minimal guarantees.
- **Brave** fails the crowd capacity test under the super pessimistic regime ($k \approx 0.008 \ll 1$), leaving returning sessions uniquely singled out across billions of empty buckets. Under the slightly pessimistic regime ($\delta = 2$), k rises to 822. Even in the slightly optimistic regime ($\delta = 2.5$), Brave reaches $k \approx 8,177$ —still about $18\times$ smaller than the absolute minimum Tor’s anonymity set despite Brave having an installed base $40\times$ larger (80 million vs. 2 million).
- **Stock Browsers (Chrome and Firefox)** leave users completely isolated at $\delta = 1$ ($k \approx 8 \times 10^{-6}$

and 4×10^{-7}). At $\delta = 2$, Firefox provides virtually no crowd protection ($k \approx 9$), while Chrome reaches $k \approx 167$. In the slightly optimistic case ($\delta = 2.5$), Chrome reaches $k \approx 4,854$ and Firefox $k \approx 250$, driven purely by global adoption size rather than any architectural entropy suppression.

- **Firefox RFP** leaves 28.73 bits of residual entropy. Hiding an RFP user within a crowd ($k \geq 1$) would require an adoption pool of $2^{H_{\text{res}}/\delta}$: 445 million users under super-pessimistic ($\delta = 1$, exceeding the entire worldwide desktop Firefox population), $\approx 21,100$ users under slightly pessimistic ($\delta = 2$), and $\approx 2,880$ users in the slightly optimistic case ($\delta = 2.5$). In figure 3 we estimate an absolutely optimistic N of 1 million users

These findings highlight that population size is a primary security parameter: uniformity leverages collective adoption to create expansive anonymity, whereas randomization leaves devices isolated even under generous correlation models.

4.6 An Adversary Who Discards the Noise

While a naive adversary hashes all attributes for exact equality, an **adaptive adversary** can detect the browser type and selectively discard its farbled channels. Defenses differ markedly in the breadth of their noise: Firefox RFP perturbs a minimal surface (primarily `canvas` and `audio`), whereas Brave randomizes a broader set of vectors (including `screenResolution` and `hardwareConcurrency`).

By pruning these known unstable attributes, an attacker eliminates variability and re-establishes deterministic **exact hash matching** on the surviving static properties. For Firefox RFP, simply omitting `canvas` the leaked entropy will be 100% static. For Brave, discarding all farbled channels (that are slightly more e.g. `audio`, `hardwareConcurrency`) leads to the same result (fingerprint *stability*). Conversely, **Tor** remains impervious to selective pruning: its privacy stems from universal pinning rather than volatile noise, maintaining an 88%–93% entropy reduction and preserving large anonymity crowds.

Fuzzy Similarity Matching. An adversary does not even need to drop farbled attributes; they can replace exact equality with similarity matching and possibly recover some entropy. This is the exact risk that firefox RFP and Tor want to avoid.

When comparing the fraction of the 43 attributes that match between independent sessions on the same host, Brave sessions share an average agreement of 0.906 (minimum $38/43 \approx 0.884$). The closest other product comparison, Chrome on the same machine, reaches only $37/43 \approx 0.860$ against Brave. This separates browser groups on this host.

A threshold of 0.87 links all 780 Brave pairs and none of the compared other product pairs (Fig. 4).

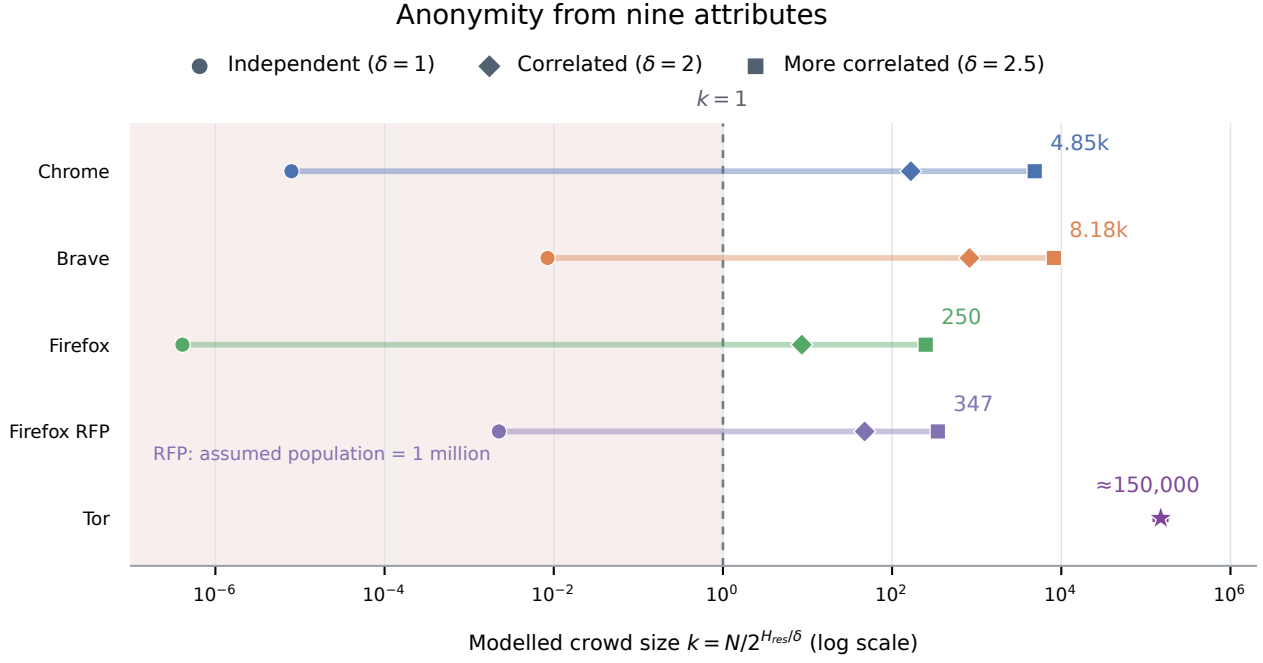


Figure 3: Modelled anonymity from nine entropy weighted attributes under three deflation scenarios. Tor uses the separate enumeration described in the text.

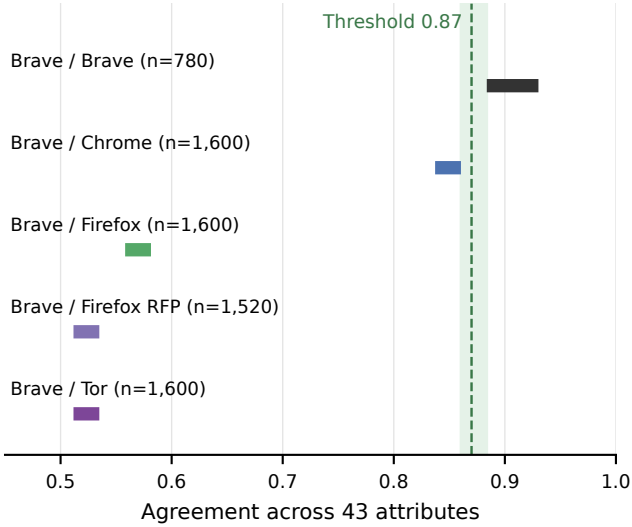


Figure 4: Observed agreement ranges on one host. A threshold of 0.87 separates Brave session pairs from the other product groups, while exact hashes match no Brave pair. The aggressive setting is excluded as the same product family.

4.7 Cross-Browser Linkability

Finally, we evaluate whether commercial JavaScript fingerprinting can track a user switching between different browsers on the same physical host. Across the 17 discriminative, farbling-immune components, agreement clusters strictly by *engine family* rather than physical machine: intra-engine agreement reaches 0.94 within Chromium (Chrome vs. Brave) and 0.47–0.82 within Gecko, but collapses to merely 0.06–0.24 across engines (Chrome vs. Firefox).

Cao et al. [5] demonstrated that cross browser linkage is feasible on undefended browsers. While modern API mitigations complicate purely client-side attacks, combining JavaScript attributes with external markers—such as network and TLS signatures (Section 6) or behavioral patterns—renders cross-browser tracking a tangible and effective threat. These techniques paired with careful calibration of similarity thresholds (Fig. 4), and possibly paired with machine learning classifiers, could nonetheless help identify the same host across browser boundaries.

5 DISCUSSION

Synthesizing our empirical results yields three central conclusions regarding browser defense viability, tracking mechanics, and experimental limits.

Tor as the Only Robust Defense. Across all evaluated metrics, Tor Browser emerges as the only defense that withstands JavaScript fingerprinting in our tests. By suppressing residual entropy to $H_{\text{res}} = 4.65$ bits, Tor places every user into an expansive anonymity set: its two surviving attributes enumerate to 12–15 populated hardware buckets, so that even the rarest configuration retains $k \approx 10^4$ peers and common ones exceed 4×10^5 . This is an estimate of the *worst* case under an assumed hardware mix, not a mean, and it renders the fingerprint useless for individual identification. This is also evidenced by the fact that Tor removed OS spoofing, which was introducing detectable cross attribute anomalies that increased rather than reduced residual entropy.

Partial Uniformity: The Niche Fingerprint. Our findings demonstrate that partial uniformity is not merely an incomplete defense, but actively counterproductive.

Standardization protects solely by concealing the user within an expansive, homogeneous crowd. However, Firefox RFP commands a minuscule user base ($N_{\text{RFP}} \ll N_{\text{Firefox}}$). By standardizing only a subset of attributes (such as pinning the timezone to `Atlantic/Reykjavik` and color depth to 24) while leaving high-entropy system properties unmasked (28.73 bits across `userAgent`, `fonts`, `languages`, `hardwareConcurrency`, and `platform`), RFP fails to blend the user into the crowd. Instead, it yields anonymity sets comparable to or worse than stock Firefox, all while inflicting severe usability penalties. Furthermore, this selective standardization creates a specific *niche fingerprint* through unnatural cross attribute anomalies. In our testbed, for example, the host’s Italian system language (`it-IT`) was leaked unmodified alongside its physical CPU architecture, yet paired with an Icelandic timezone (`Atlantic/Reykjavik`). An adaptive tracker immediately deduces that the client is an RFP user operating from Italy; within that tiny demographic slice, the surviving 28.73 bits of unmasked entropy readily single out the device. This fosters a dangerous false sense of security and illustrates the classic “standing-out” paradox (or *grey sheep* effect): by attempting to adopt the uniform of a crowd that does not exist, RFP users become far more conspicuously trackable than mainstream users who naturally blend into the dominant population.

Brave’s defense is purely *cosmetic* rather than structural: instead of altering the device, it merely injects ephemeral noise into five specific JavaScript APIs. While this disrupts naive exact-hash matching across visits, the underlying hardware and its remaining 38 unperturbed attributes remain completely static and unique. An adaptive adversary readily bypasses this superficial disguise by either: (1) **pruning the noisy channels**, which instantly recovers a stable and unique host fingerprint, or (2) **fuzzy matching** (similarity threshold 0.87), which reliably links every session by exploiting the stability of the 38 unchanged attributes. In short, randomization merely fools rigid equality tests; genuine anonymity requires structural uniformity that dissolves uniqueness within a crowd.

Threats to Validity and Experimental Improvements. Our evaluation involves four methodological boundaries, pointing directly toward concrete experimental improvements:

- **Single host baseline:** All measurements were gathered on a single Apple Silicon host on a single trial without considering temporal evolution. Heterogeneous multi-host and temporal evolving testing is the natural next step.
- **Atypical attribute pairs:** Additive marginal sums assume attribute independence; our empirical deflation factor ($\delta \approx 2.33$) estimates this joint redundancy. However, rare host pairings (e.g., `en-US` locale with `Europe/Rome`) exhibit negative *pointwise* mutual information, elevating individual surprisal beyond population averages. Furthermore we have to consider

that our entropy calculations were approximations.

- **Network-layer and behavioral tracking:** Our evaluation assessed only client side JavaScript attributes, representing a conservative lower bound on tracking capabilities. While Tor uniquely guarantees a structural anonymity floor and alternatives like Brave fail to provide minimal crowd guarantees, real world trackers rarely operate on JavaScript in isolation. Adversaries routinely fuse client-side attributes with orthogonal tracking modalities: (1) transport-layer signatures such as TLS `ClientHello` hashes (JA3/JA4) and HTTP/2 framing; (2) network identifiers like IP addresses and subnets (which in addition with fingerprinting means can isolate devices within corporate or university LANs); and (3) behavioral biometrics, including mouse trajectories and keystroke dynamics. Combining client side fingerprints with these external vectors multiplies identifying entropy, drastically amplifying tracking power even against noisy browsers. Finally, because our testbed was deployed over local HTTP, capturing JA3/JA4 signatures and empirically distinguishing Tor’s *Safer* and *Safest* modes at the transport layer requires introducing a TLS-terminating HTTPS proxy, forming a natural direction for future experimental work.

6 ADDITIONAL CONSIDERATIONS

In production, web fingerprinting operates far beyond isolated client-side scripts: trackers routinely evade ad-blockers and privacy audits by deploying *first-party reverse proxies* and *network-layer signatures* (e.g., TLS JA3/JA4 hashes), using device attributes primarily as *regeneration anchors* [11] to silently respawn deleted cookies. This architectural opacity directly collides with European data protection law. While tracking vendors argue that fingerprints collect no *Personally Identifiable Information* (PII), European law rejects this distinction: the GDPR (Art. 4(1)) regulates any telemetry enabling individual *singling out* (personal data), and ePrivacy Art. 5(3) [9] strictly mandates prior opt-in consent for querying terminal hardware, whether stateful or ephemeral. Our benchmark maps directly onto these legal priorities: **Tor’s uniformity** defeats *identifiability* by hiding users within massive anonymous crowds ($k \gg 1$), whereas **Brave’s randomization** defeats *linkability* by salting noise with the domain ($K = \text{session_key} \oplus \text{domain_key}$) to cryptographically block cross-site profiling independent of publisher consent compliance.

7 CONCLUSION

Browser fingerprinting succeeds when device identifiers are simultaneously *unique* and *temporally stable*. Our empirical benchmark demonstrates that the two prevailing defenses address this conjunction asymmetrically: **Tor’s structural uniformity** dismantles uniqueness by standardizing the runtime environment, suppressing residual

entropy to 4.65 bits and placing users into expansive anonymity sets ($k \approx 1.5 \times 10^5$). In contrast, **Brave’s randomization** attempts to disrupt stability via client-side noise, while conceding the host’s underlying uniqueness (33.14 bits of unmasked entropy). Against an adaptive adversary who filters noisy channels or applies fuzzy similarity matching (threshold 0.87), farbling’s protection evaporates (0% reduction on surviving attributes), demonstrating that ephemeral noise only defeats naive equality hashing. Meanwhile, **Firefox RFP** exposes the hazard of partial uniformity and small user bases: standardizing isolated attributes within a small user base might create an anomalous “grey sheep” fingerprint that singles users out rather than blending them into the crowd. Ultimately, generic JavaScript fingerprinting remains a potent tracking threat even though it’s not easily exploitable. The exploitation of these tools with more potent network and behavioural layer fingerprinters should worry security vendors, regulators, and the users themselves.

Finally, we admit that simple JS fingerprinting might still be dangerous in 2026, despite the great number of defenses that exist on browsers. Crucially, randomization offers a pragmatic compromise for everyday web usability (Fig. 5), but only the well implemented uniformity and the protections provided by the Tor network can assure secure web browsing.

REFERENCES

- [1] Alex Berke, Enrico Bacis, Badih Ghazi, Pritish Kamath, Ravi Kumar, Robin Lassonde, Pasin Manurangsi, and Umar Syed. How unique is whose web browser? the role of demographics in browser fingerprinting among us users. *Proceedings on Privacy Enhancing Technologies*, 2025(1), 2025.
- [2] Brave Software. Fingerprinting protections. <https://github.com/brave/brave-browser/wiki/Fingerprinting-Protections>, 2024.
- [3] Brave Software. brave-core source code: `brave_session_cache.cc`, `chrome_client_impl.cc`. <https://github.com/brave/brave-core>, 2026. Accessed September 2026.
- [4] Brave Software. Brave improves protections against GPU fingerprinting. <https://brave.com/privacy-updates/38-webgl-webgpu-fingerprinting-protections/>, 2026.
- [5] Yinzhi Cao, Song Li, and Erik Wijmans. (cross-)browser fingerprinting via os and hardware level features. In *Network and Distributed System Security Symposium (NDSS)*, 2017.
- [6] Yana Dimova, Gunes Acar, Lukasz Olejnik, Wouter Joosen, and Tom Van Goethem. The CNAME of the game: Large-scale analysis of DNS-based tracking evasion. *Proceedings on Privacy Enhancing Technologies*, 2021(3):394–412, 2021.
- [7] Peter Eckersley. How unique is your web browser? In *Privacy Enhancing Technologies (PETS)*, pages 1–18, 2010.
- [8] Steven Englehardt and Arvind Narayanan. Online tracking: A 1-million-site measurement and analysis. In *ACM CCS*, pages 1388–1401, 2016.
- [9] European Data Protection Board. Guidelines 2/2023 on technical scope of art. 5(3) of the eprivacy directive. Version 2.0, adopted 7 October 2024, 2024.
- [10] FingerprintJS Inc. FingerprintJS v5.2.0. <https://github.com/fingerprintjs/fingerprintjs>, 2025.
- [11] Imane Fouad, Cristiana Santos, Arnaud Legout, and Nataliaia Bielova. My cookie is a phoenix: Detection, measurement, and lawfulness of cookie respawning with browser fingerprinting. *Proceedings on Privacy Enhancing Technologies*, 2022(3):79–98, 2022.
- [12] Alejandro Gómez-Boix, Pierre Laperdrix, and Benoit Baudry. Hiding in the crowd: An analysis of the effectiveness of browser fingerprinting at large scale. In *The Web Conference (WWW)*, pages 309–318, 2018.
- [13] Pierre Laperdrix, Nataliaia Bielova, Benoit Baudry, and Gildas Avoine. Browser fingerprinting: A survey. *ACM Transactions on the Web*, 14(2):1–33, 2020.
- [14] Pierre Laperdrix, Walter Rudametkin, and Benoit Baudry. Beauty and the beast: Diverting modern web browsers to build unique browser fingerprints. In *IEEE Symposium on Security and Privacy*, pages 878–894, 2016.
- [15] Keaton Mowery and Hovav Shacham. Pixel perfect: Fingerprinting canvas in html5. In *Web 2.0 Security and Privacy (W2SP)*, 2012.
- [16] Mozilla Corporation. Firefox source code: `ImageExtractionResult()` and `GeneratePlaceholderCanvasData()`. mozilla-central, `dom/canvas/`, <https://searchfox.org/mozilla-central/source/dom/canvas/GeneratePlaceholderCanvasData.h>, 2026. Accessed September 2026.
- [17] Hoang Dai Nguyen and Phani Vadrevu. Breaking the shield: Analyzing and attacking canvas fingerprinting defenses in the wild. In *Proceedings of the ACM Web Conference (WWW ’25)*, Sydney, NSW, Australia, 2025.
- [18] Gaston Pugliese, Christian Riess, Freya Gassmann, and Zinaida Benenson. Long-term observation on browser fingerprinting: Users’ trackability and perspective. *Proceedings on Privacy Enhancing Technologies*, 2020(2):558–577, 2020.
- [19] The Tor Project. The design and implementation of the Tor browser. <https://gitlab.torproject.org/tpo/applications/wiki>, 2024.

A CANVAS EXPORT USABILITY

Figure 5 loads the same page in Brave and Tor Browser. Each canvas appears as painted on screen (left pane) and as `toDataURL()` returns it to the script (right pane); only the right pane is readable by a script, whether a tracker or a legitimate application.

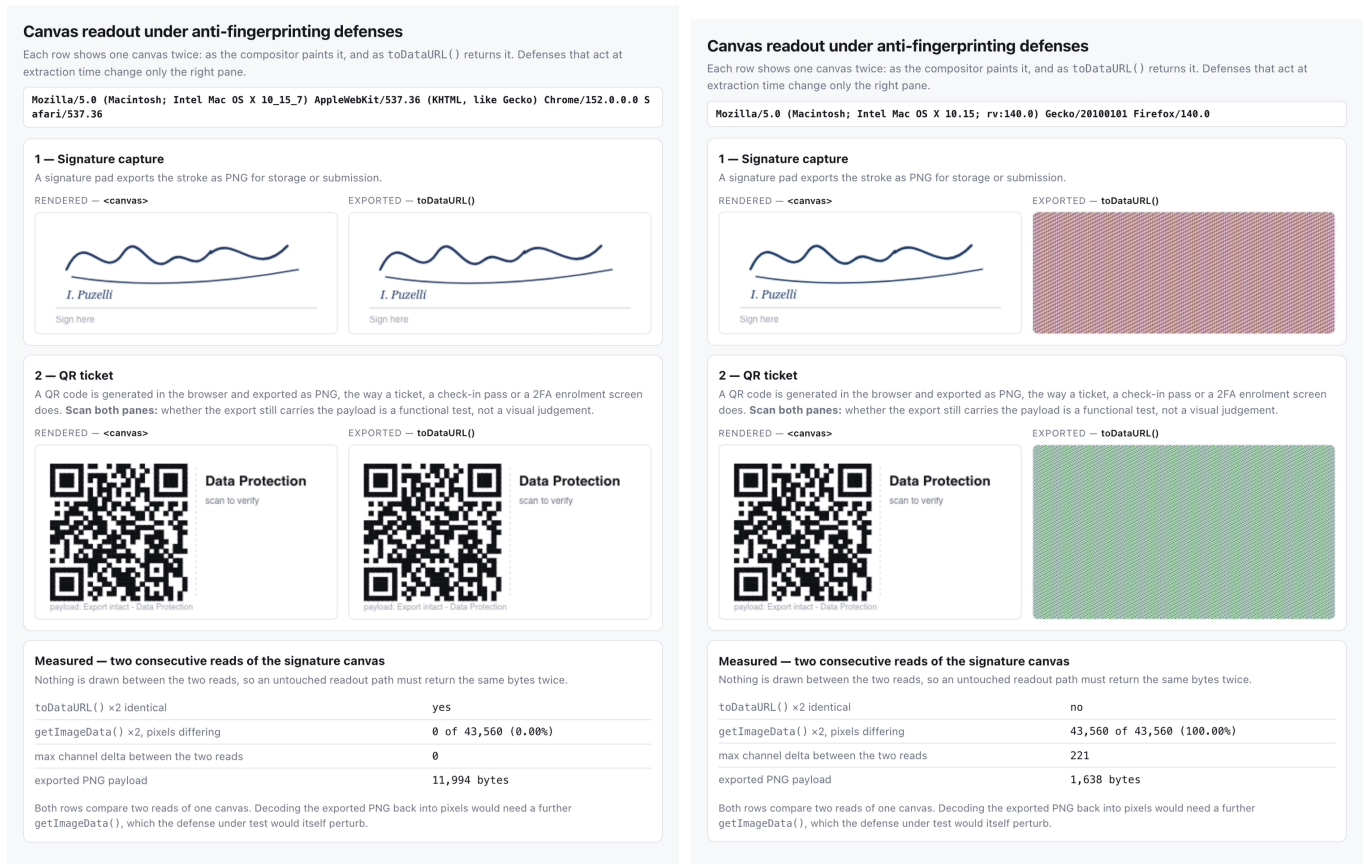


Figure 5: Canvas readout in Brave (left) and Tor Browser (right). Brave’s export matches the painted canvas and its QR code still carries the payload (841/841 modules). Tor returns a repeating pattern instead: the QR matches 432/841 modules, about chance, and two consecutive reads differ on every pixel.